



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 05

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 20226-2

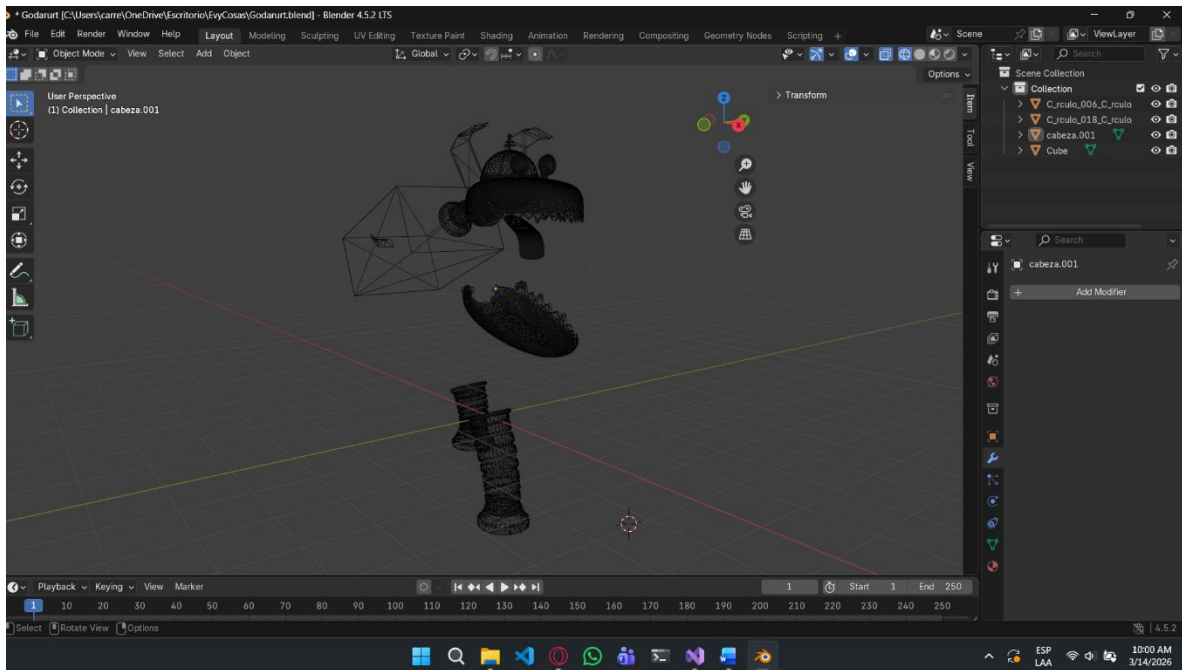
FECHA DE ENTREGA LÍMITE: 14 de marzo del 2026

CALIFICACIÓN: _____

EJERCICIO REALIZADO

Optimización y carga de modelo Goddard

Para esta actividad se pidió realizar la optimización y carga de un modelo de Goddard, proporcionado por el profesor en formato .obj, para ello, se utilizo Blender v5.0, para separar el modelo en partes, siendo estas una patita delantera, una patita trasera, la mandíbula inferior y el resto del cuerpo.



Se guardaron estos modelos de forma individual obteniendo por resultado los siguientes archivos.

Nombre	Fecha de modificación	Tipo	Tamaño
 PatitaDelantera1.mtl	3/13/2026 3:34 PM	Archivo MTL	1 KB
 PatitaDelantera1.obj	3/13/2026 3:34 PM	Object File	488 KB
 PatitaDelantera2.mtl	3/13/2026 3:34 PM	Archivo MTL	1 KB
 PatitaDelantera2.obj	3/13/2026 3:34 PM	Object File	488 KB
 PatitaTrasera2.mtl	3/13/2026 3:24 PM	Archivo MTL	1 KB
 PatitaTrasera2.obj	3/13/2026 3:24 PM	Object File	483 KB
 PatitaTrasera1.mtl	3/13/2026 3:24 PM	Archivo MTL	1 KB
 PatitaTrasera1.obj	3/13/2026 3:24 PM	Object File	483 KB
 mandibula.mtl	3/13/2026 3:17 PM	Archivo MTL	1 KB
 mandibula.obj	3/13/2026 3:17 PM	Object File	353 KB
 cuerpito.mtl	3/13/2026 3:11 PM	Archivo MTL	1 KB
 cuerpito.obj	3/13/2026 3:11 PM	Object File	2,018 KB

Pasando ahora a Visual Studio – OpenGL, los cambios realizados son los siguientes.

Window.h
Definimos las funciones getter publicas dentro de <code>class Window</code> , esto para que nos puedan permitir que otros archivos usen las articulaciones.
<pre>// ===== // GETTERS: Articulaciones de Goddard GLfloat getarticulacion1() { return articulacion1; } // PataDelanteraDerecha (D1) GLfloat getarticulacion2() { return articulacion2; } // PataDelanteraIzquierda (D2) GLfloat getarticulacion3() { return articulacion3; } // PataTraseraDerecha (T1) GLfloat getarticulacion4() { return articulacion4; } // PataTraseraIzquierda (T2) GLfloat getarticulacion5() { return articulacion5; } // MandibulaInferior // =====</pre>
Dentro de nuestra sección privada, declaramos las variables de tipo GLfloat, para almacenar los valores de grados.
<pre>// ===== // VARIABLES: Control de ángulos de rotación (Goddard) GLfloat articulacion1, articulacion2, articulacion3, articulacion4, articulacion5; // =====</pre>

Window.cpp
Inicializamos las variables de articulación en la sección de <code>Window::Window(GLint win- dowWidth, GLint windowHeight)</code>
<pre>// ===== articulacion1 = 0.0f; articulacion2 = 0.0f; articulacion3 = 0.0f; articulacion4 = 0.0f; articulacion5 = 0.0f; // =====</pre>
Para nuestra lógica de articulaciones dentro de <code>void Window::ManejaTeclado(...)</code> manejamos las teclas de F, G, H y J para mover las piernas hacia adelante, mientras que las teclas V, B, N y M serán para mover las patitas hacia atrás. Por otro lado, las teclas O y L servirán para mover la mandíbula, todas estas mediane un tope de +45° y -45° como se aplico en la práctica anterior.
<pre>// ===== // ANIMACIONES DE GODDARD: (Límites: +45° a -45°) // Pata Delantera Derecha (D1) - Teclas F (Adelante) y V (Atrás) if (key == GLFW_KEY_F) { theWindow->articulacion1 += 2.0f; // Velocidad de rotación if (theWindow->articulacion1 > 45.0f) theWindow->articulacion1 = 45.0f; } if (key == GLFW_KEY_V) { theWindow->articulacion1 -= 2.0f; if (theWindow->articulacion1 < -45.0f) theWindow->articulacion1 = -45.0f; } }</pre>

```

// Pata Delantera Izquierda (D2) - Teclas G (Adelante) y B (Atrás)
if (key == GLFW_KEY_G) {
    theWindow->articulacion2 += 2.0f;
    if (theWindow->articulacion2 > 45.0f) theWindow->articulacion2 = 45.0f;
}
if (key == GLFW_KEY_B) {
    theWindow->articulacion2 -= 2.0f;
    if (theWindow->articulacion2 < -45.0f) theWindow->articulacion2 = -45.0f;
}

// Pata Trasera Derecha (T1) - Teclas H (Adelante) y N (Atrás)
if (key == GLFW_KEY_H) {
    theWindow->articulacion3 += 2.0f;
    if (theWindow->articulacion3 > 45.0f) theWindow->articulacion3 = 45.0f;
}
if (key == GLFW_KEY_N) {
    theWindow->articulacion3 -= 2.0f;
    if (theWindow->articulacion3 < -45.0f) theWindow->articulacion3 = -45.0f;
}

// Pata Trasera Izquierda (T2) - Teclas J (Adelante) y M (Atrás)
if (key == GLFW_KEY_J) {
    theWindow->articulacion4 += 2.0f;
    if (theWindow->articulacion4 > 45.0f) theWindow->articulacion4 = 45.0f;
}
if (key == GLFW_KEY_M) {
    theWindow->articulacion4 -= 2.0f;
    if (theWindow->articulacion4 < -45.0f) theWindow->articulacion4 = -45.0f;
}

// Mandíbula (Hocico) - Teclas O (Abrir) y L (Cerrar)
if (key == GLFW_KEY_O) {
    theWindow->articulacion5 += 2.0f;
    // Tope de apertura máxima configurada en 20 grados
    if (theWindow->articulacion5 > 20.0f) theWindow->articulacion5 = 20.0f;
}
if (key == GLFW_KEY_L) {
    theWindow->articulacion5 -= 2.0f;
    // Tope de cierre (evita que traspase el cráneo, límite en -25 grados)
    if (theWindow->articulacion5 < -25.0f) theWindow->articulacion5 = -25.0f;
}
// =====

```

Main (E05-317314975)

Declaramos las instancias globales de la clase Model, cada una representando una parte independiente de Goddard.

```

// =====
Camera camera;
Model Goddard_M;
Model Cuerpito;
Model Hocico;
Model PatitaD1;
Model PatitaD2;
Model PatitaT1;
Model PatitaT2;
// =====

```

Cargamos los modelos con el método LoadModel para cada una de nuestras instancias, este método es gracias a nuestra librería Assimp que lee archivos .obj, justo como los modelos que exportamos de blender.

```
// =====
Goddard_M = Model();
Goddard_M.LoadModel("Models/goddard_base.obj");
Cuerpito = Model();
Cuerpito.LoadModel("Models/Cuerpito.obj");
Hocico = Model();
Hocico.LoadModel("Models/mandibula.obj");
PatitaD1 = Model();
PatitaD1.LoadModel("Models/PatitaDelantera1.obj");
PatitaD2 = Model();
PatitaD2.LoadModel("Models/PatitaDelantera2.obj");
PatitaT1 = Model();
PatitaT1.LoadModel("Models/PatitaTrasera1.obj");
PatitaT2 = Model();
PatitaT2.LoadModel("Models/PatitaTrasera2.obj");
// =====
```

Optimizamos los colores, definiendo vectores antes de entrar al ciclo while principal para que no se creen en cada ciclo de reloj.

```
// =====
glm::mat4 model(1.0);
glm::mat4 modelaux(1.0);
glm::vec3 color = glm::vec3(1.0f, 1.0f, 1.0f);
// OPTIMIZACIÓN: Instanciamos los colores UNA SOLA VEZ antes del ciclo While
glm::vec3 colorPiso = glm::vec3(0.5f, 0.5f, 0.5f); // Gris
glm::vec3 colorCuerpo = glm::vec3(0.2f, 0.2f, 0.2f); // Gris oscuro
glm::vec3 colorHocico = glm::vec3(0.8f, 0.1f, 0.1f); // Rojo
glm::vec3 colorPatitaD1 = glm::vec3(0.1f, 0.8f, 0.1f); // Verde
glm::vec3 colorPatitaD2 = glm::vec3(0.1f, 0.1f, 0.8f); // Azul
glm::vec3 colorPatitaT1 = glm::vec3(0.8f, 0.8f, 0.1f); // Amarillo
glm::vec3 colorPatitaT2 = glm::vec3(0.8f, 0.1f, 0.8f); // Morado
// =====
```

Hacemos nuestro modelo renderizado, partiendo de la parte del cuerpo principal ya trabajada en el laboratorio, el nodo padre será el “cuerpito” instanciando la matriz identidad y después, haciendo la traslación global a arriba (en Y) del suelo instanciado por el profesor. Pasamos a hacer el respaldo en modelaux y aplicamos los nodos hijos para cada parte restante de patas y mandíbula aplicando solo cambios de traslación para posicionarlos mejor y cambios de rotación dependientes a las teclas declaradas anteriormente.

```
// =====
// JERARQUÍA DE GODDARD

// -----
// 1. NODO PADRE: Cuerpito (Cuerpo principal)
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -0.1f, -1.5f));
modelaux = model; // GUARDAMOS PIVOTE para que hereden los hijos
glUniform3fv(uniformColor, 1, glm::value_ptr(colorCuerpo));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cuerpito.RenderModel();

// -----
```

```

// NODO HIJO 1: Mandibula inferior (Hocico)
model = modelaux;
// Traslación relativa
model = glm::translate(model, glm::vec3(2.7f, 1.4f, 0.0f));
// Rotación sobre el eje Z para abrir/cerrar el hocico (Teclas O y L)
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorHocico));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Hocico.RenderModel();

// -----
// NODO HIJO 2: Pata delantera derecha (D1)
model = modelaux;
model = glm::translate(model, glm::vec3(0.8f, 0.27f, 0.6f));
// Rotación sobre el eje Z para caminar (Teclas F y V)
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPatitaD1));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
PatitaD1.RenderModel();

// -----
// NODO HIJO 3: Pata delantera izquierda (D2)
model = modelaux;
model = glm::translate(model, glm::vec3(0.8f, 0.27f, -0.6f));
// Rotación sobre el eje Z para caminar (Teclas G y B)
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPatitaD2));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
PatitaD2.RenderModel();

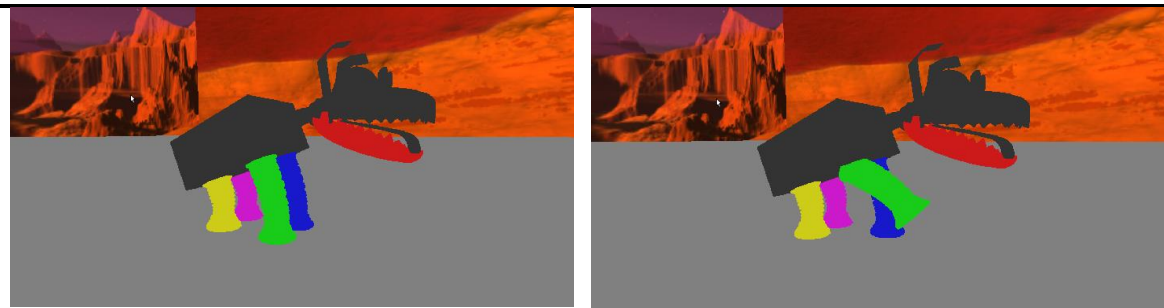
// -----
// NODO HIJO 4: Pata trasera derecha (T1)
model = modelaux;
model = glm::translate(model, glm::vec3(-0.85f, -0.4f, 0.6f));
// Rotación sobre el eje Z para caminar (Teclas H y N)
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPatitaT1));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
PatitaT1.RenderModel();

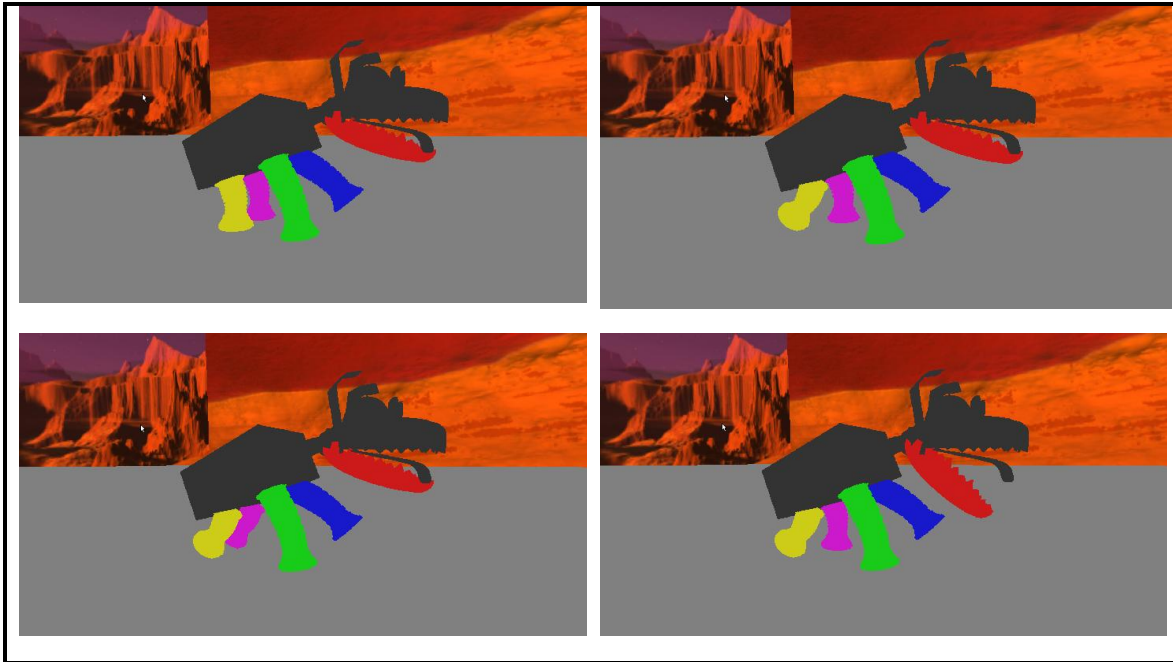
// -----
// NODO HIJO 5: Pata trasera izquierda (T2)
model = modelaux;
model = glm::translate(model, glm::vec3(-0.85f, -0.4f, -0.6f));
// Rotación sobre el eje Z para caminar (Teclas J y M)
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
glUniform3fv(uniformColor, 1, glm::value_ptr(colorPatitaT2));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
PatitaT2.RenderModel();
// =====

```

EJECUCIÓN

Teclas de movimiento: F,G,H,J – V,B,N,M – O,L





Para una mejor visualización se adjunta un link al GIF de drive.
https://drive.google.com/file/d/1vmJ_4giywcox-WO6W6FfG-St_ma3I2Ei/view?usp=sharing

PROBLEMAS PRESENTADOS

Durante el desarrollo de esta práctica no se presentaron complicaciones mayores. La lógica de programación y la implementación de la jerarquía de nodos resultaron ser muy similares al trabajo realizado en la práctica anterior con el modelo de Catnap, lo que agilizó bastante el flujo de trabajo. Adicionalmente, gracias a que ya se contaba con experiencia básica utilizando el software de modelado Blender, la tarea de importar el modelo original de Goddard, separar sus piezas de forma independiente (cuerpo, mandíbula y patas) y exportarlas con los pivotes ubicados en el origen no representó una dificultad significativa.

CONCLUSIONES

Sobre el ejercicio de clase

La complejidad de esta práctica fue sencilla a nivel de código, pero requirió de un mayor razonamiento espacial y matemático. Si bien la lógica de las transformaciones jerárquicas ya se había explorado en prácticas anteriores (como en el modelo de nuestra mascota), la transición de usar primitivas básicas a importar modelos tridimensionales complejos (.obj) elevó el nivel de exigencia. La explicación de la clase cubrió satisfactoriamente las bases

teóricas de la librería Assimp y ya con ello, se visualiza de mejor manera el trabajo a realizar próximamente en el importado de modelos externos para nuestro proyecto final.

Comentarios generales

Como sugerencia para futuras prácticas de involucren Blender, sería de gran ayuda volver a proporcionar material de apoyo. No es necesario crear el material desde cero; bastaría con proporcionar enlaces a tutoriales de Blender ya existentes en internet que expliquen paso a paso trucos que nos pueden ahorrar tiempo en la manipulación de modelos, personalmente se encontraron algunos tips y atajos que ahorro bastante tiempo para la separación del modelo de Goddard, ajustar los pivotes al origen coordenado y exportar correctamente a formato .obj.